Date	15 March 2026
Team ID	XXXXXX
Project Name	VBCUA — Voice-Based Concept Understanding Analyser
Maximum Marks	5 Marks

CODING & SOLUTION

CODING & SOLUTION DETAILS:

This section evaluates the quality and completeness of your implemented code and the overall solution. Your submission should demonstrate working functionality, clean code practices, and alignment with your defined requirements.

SOLUTION SUMMARY

Repository Link / URL	`https://github.com/user/vbcua-voice-analyser`
Programming Language(s)	Python
Framework(s) Used	Streamlit Framework
Key Features Implemented	• **Streamlit Graphical Interface:** Two-tab system layout implementing real-time audio file uploading components and visual timeline data grids. • **Automated Transcription Pipe:** Local OpenAI Whisper translation routing to translate speech data into string blocks. • **Semantic Vector Embeddings:** Local Sentence-Transformers checking (60% weight criteria) computing conceptual compliance metrics. • **Acoustic Fluency Matrix:** Librosa sound signal feature matching (40% weight criteria) to tally speech duration, fillers, and words-per-minute (WPM). • **Generative AI Feedback:** Google Gemini API synthesis generating contextual criticism notes alongside local SQLite standalone tracking storage.
Pending / Incomplete Features	None. Full baseline specification engine implemented successfully end-to-end.

Setup / Run Instructions	`1. pip install -r requirements.txt`
	`2. cp .env.example .env (Configure keys)`
	`3. streamlit run app.py`

CODE QUALITY CHECKLIST

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

ADDITIONAL NOTES / COMMENTS

The system has been successfully verified on local test runs. It includes a smart fallback rule-based mechanism that triggers to ensure scoring execution runs smoothly even if external LLM api keys are omitted or if network connectivity is completely disconnected. Code layouts follow proper clean-code conventions.